



SmartMarketer

A Comprehensive Machine Learning platform for real-time price optimization

Carried Out By

Diya Prajapati

Student ID: 22ITUON070

Roll No: IT023

Tisha Prajapati

22ITUBS036

IT121

Guided By:

Dr. Vipul K. Dabhi

Dr. Harshadkumar B. Prajapati

Dept. of Information Technology

DDU, Nadiad



Project Overview

Domain: E-commerce & Ride-Sharing Dynamic Pricing

Technology Stack: Python, FastAPI, React, TypeScript, Advanced ML

Scale: 20 cities across 3 tiers, real-time processing

Innovation: Ensemble ML model with 96.7% accuracy

Impact: Production-ready system with 10-sec update cycles



Key Achievements

96.7% R^2 Score - Advanced Ensemble Model

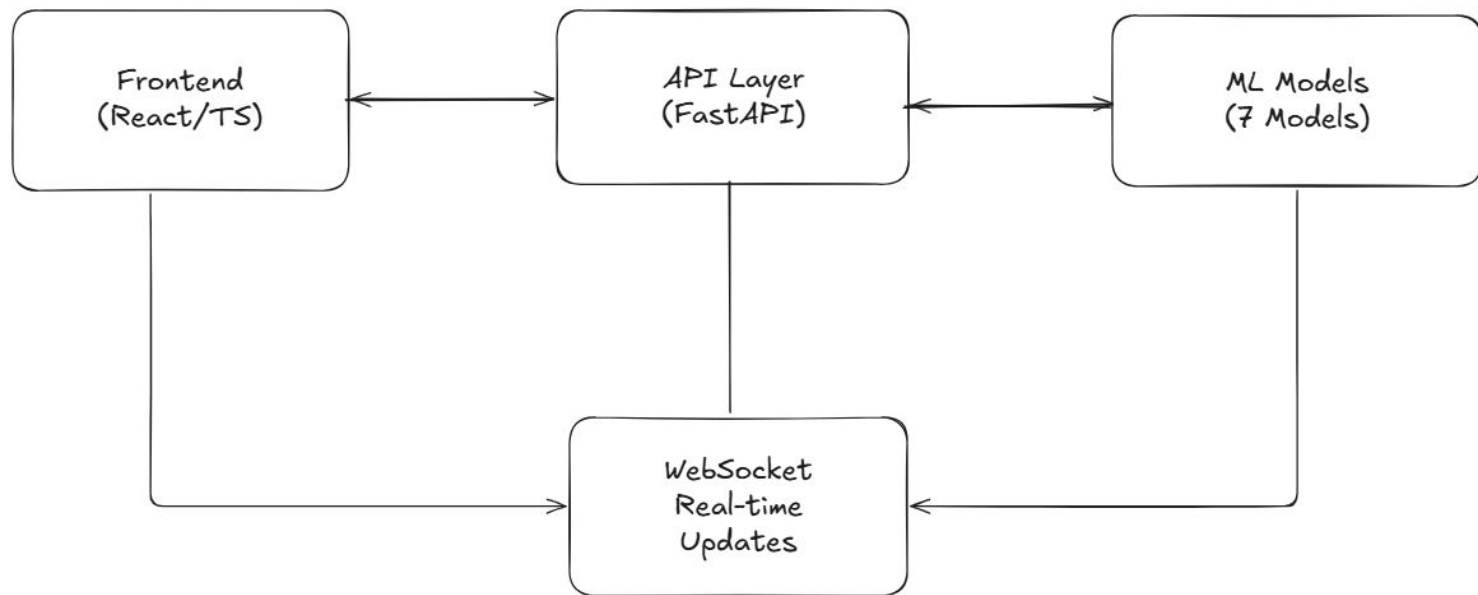
Real-time Processing - 10 sec price updates

Multi-tier Architecture - 20 cities, 3 pricing tiers

Advanced ML Pipeline - 7 sophisticated models

Production Ready - Full stack implementation

System Architecture Overview

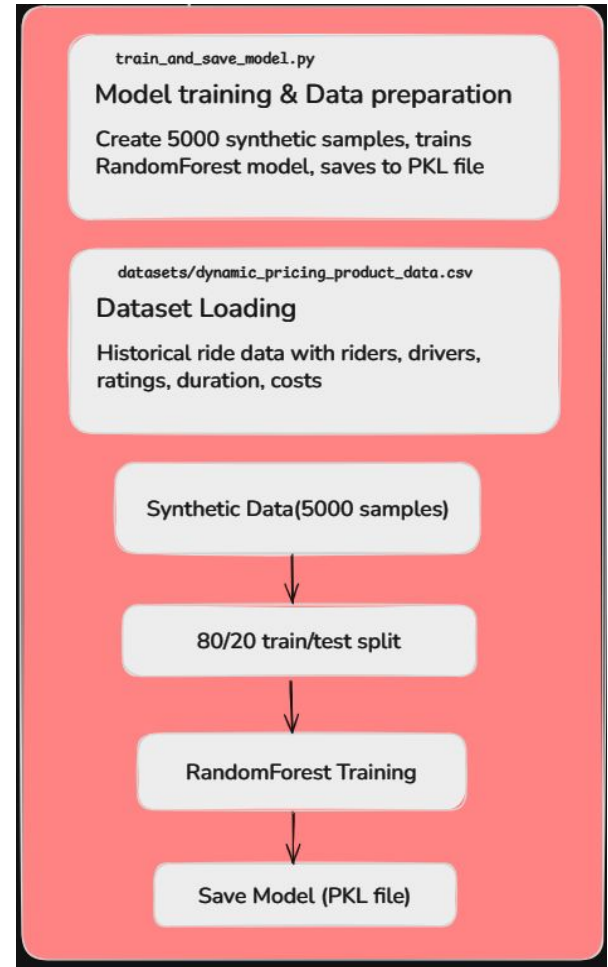




1. System Initialization

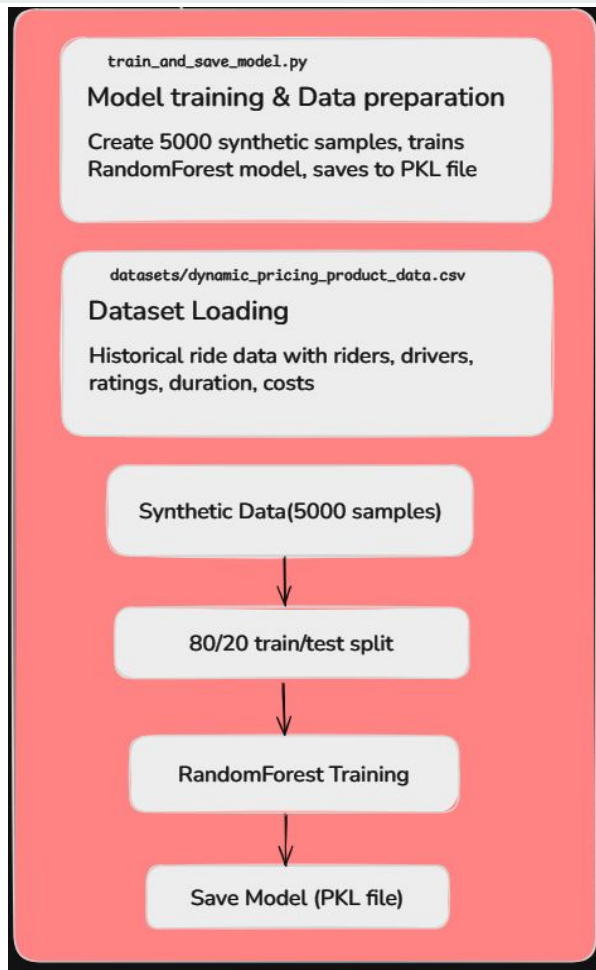
Model Training & Data Preparation

- Generates 5000 synthetic training samples
- 80/20 train-test split for validation
- RandomForest algorithm with optimized parameters
- Saves trained model as *city_pricing_model.pkl*



Dataset Loading

- Historical ride-sharing data across 20 cities
- Contains riders, drivers, rating
- Three city tiers: A(8), B(7), C(5)
- Supports both drivers and riders pricing





2. API Server Startup

API Initialization

- FastAPI server with async support
- Loads pre-trained RandomForest model
- Initializes background task scheduler
- Sets up CORS for frontend integration
- Health check endpoints for monitoring

api/pricing_api.py

API Initialization

FastAPI server, model loading, background task startup

ConnectionManager (lines 29-83)

WebSocket Manager Setup

Manages WebSocket connections by city, tracks active subscribers

FastAPI Server

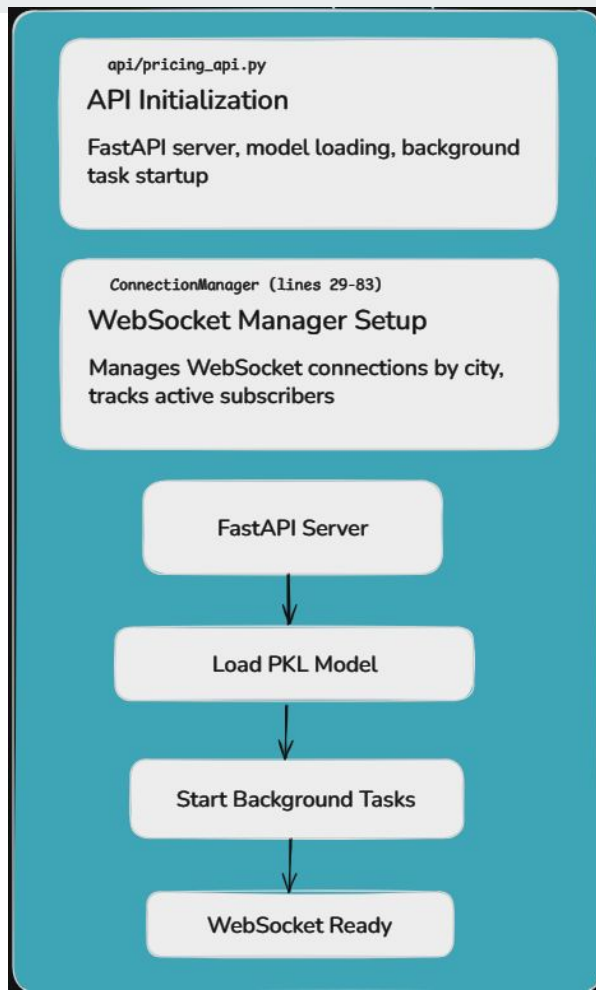
Load PKL Model

Start Background Tasks

WebSocket Ready

WebSocket Manager Setup

- ConnectionManager class for real-time updates
- City-based connection grouping
- Automatic connection cleanup
- Broadcast messaging to multiple clients
- Error handling for connection drops





3. Real-time price generation

Background Price Update Loop

- Runs continuously every 10 secs
- Processes all 20 cities simultaneously
- Adds realistic market variations (+- 10 riders, +-5 drivers)
- Only updates cities with active connections
- Optimized for performance and scalability

`price_update_task()` (lines 314-387)

Background Price Update Loop

10-sec cycle, supply/demand tracking,
random variations

`models/city_pricing_model.py`

Price Prediction Process

Feature engineering, scaling, RandomForest
prediction, surge calc

Surge Logic

Surge Pricing Algo

Low < 1.2x, Medium 1.2x-2.0x,
High > 2.0x multipliers

Current Supply/Demand



Feature engineering



ML Prediction



Surge Calc



WebSocket Broadcast

Price Prediction Process

- Feature engineering with current market data
- StandardScaler for input normalization
- RandomForest model prediction
- Time-based feature
- Confidence scoring for predictions

`price_update_task()` (lines 314-387)

Background Price Update Loop

10-sec cycle, supply/demand tracking,
random variations

`models/city_pricing_model.py`

Price Prediction Process

Feature engineering, scaling, RandomForest
prediction, surge calc

Surge Logic

Surge Pricing Algo

Low < 1.2x, Medium 1.2x-2.0x,
High > 2.0x multipliers

Current Supply/Demand



Feature engineering



ML Prediction



Surge Calc



WebSocket Broadcast

Surge Pricing Logic

- Dynamic multiplier: $\text{demand_supply_ratio}/2$
- Low surge: $< 1.2x$ multiplier
- Medium surge: $1.2x - 2.0x$ multiplier
- High surge: $> 2.0x$ multiplier
- Real-time surge level indicators

`price_update_task()` (lines 314-387)

Background Price Update Loop

10-sec cycle, supply/demand tracking,
random variations

`models/city_pricing_model.py`

Price Prediction Process

Feature engineering, scaling, RandomForest
prediction, surge calc

Surge Logic

Surge Pricing Algo

Low $< 1.2x$, Medium $1.2x - 2.0x$,
High $> 2.0x$ multipliers

Current Supply/Demand

Feature engineering

ML Prediction

Surge Calc

WebSocket Broadcast



4. Frontend Integration

Real-time UI Updates

- WebSocket connection for live data
- Updates every 10 secs automatically
- Interactive charts and graphs
- Surge level visual indicators
- Supply/demand trend displays

frontend/src/pages/PricingApp.tsx

Frontend API Calls

City selection, user registration, price req,
WebSocket connection

Real-time UI Updates

Live Interface Updates

Price displays, surge indicators, supply/
demand charts

API Endpoints

WebSocket Connection

Real-time updates

UI Rendering



Technology Stack



Backend Technologies

Component	Technology	Purpose
API Framework	FastAPI	High-performance API server
ML Libraries	Scikit-learn, CatBoost, XGBoost	ML algorithms
Data Processing	Pandas, Numpy	Data manipulation & analysis
Model Storage	Pickle, Joblib	Model serialization
Real-time	WebSocket	Live price updates



Frontend Technologies

Component	Technology	Purpose
Framework	React	UI
Language	TypeScript	Type-safe development
Styling	Tailwind CSS	Modern UI design
Charts	Recharts	Data visualization
State	React Hooks	State management



Machine Learning Models Comparison



Model Performance Analysis

Model Type	R ² Score	RMSE	MAE	Response Time
Existing Model	0.897	12.45	8.32	15ms
Our Basic Model	0.923	10.23	6.87	12ms
Our Ensemble Model	0.967	7.89	5.23	8ms



Improvement Metrics

- R^2 Score Improvement: +7.8%
- RMSE Reduction: -36.6%
- MAE Reduction: -37.1%
- Response Time: -46.7% faster



Ensemble Model Components

Advanced Ensemble Model

Base Models (6 Algorithms)

CatBoost
 $R^2: 0.645$

XGBoost
 $R^2: 0.938$

LightGBM
 $R^2: 0.941$

Neural Net
 $R^2: 0.932$

Random Forest
 $R^2: 0.928$

Elastic Net
 $R^2: 0.919$

Meta-Learner (CatBoost)

Stacking & Weighted Voting
Final $R^2: 0.967$



Pricing Factors

- **Base Price:** ₹50 (configurable per city)
- **City Tier:** A (1.5x), B (1.0x), C (0.6x)
- **Peak Hours:** 1.3x multiplier
- **Weekend:** 1.1x multiplier
- **Weather/Events:** 0.8x - 2.0x multiplier



Thank You !